# Gestión de inventario

**TIENDA TP | Informe de funcionamiento**
Algoritmos y Estructura de Datos III
15 de octubre de 2026

## 1. Objetivo del módulo

Administrar productos con identificador, nombre, cantidad disponible y precio. El sistema combina una lista enlazada para operar en memoria, SQLite para conservar los registros y una interfaz web para consultar, agregar y editar productos.

**Estado actual:** consulta, alta y edición están conectadas a la web. La eliminación está implementada en la lógica de inventario y persistencia, pero todavía no tiene ruta DELETE ni botón en pantalla.

**Navegador: HTML + CSS + JavaScript**
**↓ fetch / JSON**
**Servidor HTTP: rutas de la API**
**↓**
**Inventario: reglas y coordinación**
**↓**
**TDA Lista en memoria + persistencia SQLite**

## 2. Responsabilidades de cada archivo

| Archivo | Responsabilidad |
|---|---|
| main.py | Inicia el servidor al ejecutar el programa. |
| BACKEND/servidor.py | Sirve el frontend, recibe HTTP y devuelve JSON. |
| BACKEND/inventario.py | Coordina validaciones, TDA y persistencia. |
| BACKEND/producto.py | Construye y valida el diccionario del producto. |
| BACKEND/tad_listas.py | Encapsula los nodos, enlaces y primitivas de la lista. |
| BD/persistencia.py | Abre conexiones y ejecuta el CRUD sobre SQLite. |
| FRONTEND/ | HTML: estructura; CSS: presentación; app.js: interacción y API. |

# El producto y la lista enlazada

## 3. Representación y validación

```
{"id_producto": 3, "nombre": "Teclado",
"cantidad_disponible": 10, "precio": 35000.0}
```

crear_producto() exige un ID entero positivo, un nombre de texto no vacío, stock entero mayor o igual a cero y precio numérico finito no negativo. Elimina espacios de los extremos del nombre y devuelve el precio como float. Los booleanos no se aceptan como ID, cantidad o precio.

SQLite asigna el ID definitivo. En el alta, la lógica usa temporalmente el valor 1 para validar los otros campos; ese ID provisional no se guarda ni se inserta en el TDA.

## 4. Cómo se organiza el inventario

```
inicio → [ID 1 | siguiente] → [ID 3 | siguiente]
→ [ID 8 | None]
```

Cada nodo almacena el diccionario en info y el enlace al siguiente nodo en sig. La lista es lineal y termina en None. Se mantiene ordenada por id_producto: por eso las altas con IDs crecientes aparecen al final. El orden no depende del nombre ni del precio.

| Primitiva utilizada | Aplicación |
|---|---|
| Lista() | Crea el inventario vacío. |
| inserta1(..., "id_producto") | Inserta un diccionario en orden de ID. |
| buscar1(..., "id_producto") | Localiza el nodo del ID solicitado. |
| eliminar1(..., "id_producto") | Retira el producto y reconecta los nodos. |
| barrido(inventario) | Recorre la lista y devuelve sus elementos. |

## 5. Encapsulamiento y copias

inventario.py llama a las primitivas y no recorre ni modifica los enlaces sig. Para una consulta individual lee info del nodo devuelto por buscar1(). El barrido fue adaptado para devolver datos en lugar de imprimirlos. En el archivo actual hay dos definiciones de barrido; Python utiliza la última.

Las consultas devuelven copias de los diccionarios. Así, modificar un resultado no altera el inventario sin validación ni guardado. La copia superficial es suficiente porque los campos contienen solo texto y números.

**Costo:** buscar, insertar ordenado, eliminar y recorrer pueden requerir O(n). La carga mediante inserciones ordenadas repetidas puede llegar a $O(n^2)$. La lista se utiliza para aplicar el TDA requerido por la consigna.

# Inicio, consulta y alta

## 6. Qué ocurre al iniciar

1. main.py llama a iniciar_servidor().
2. cargar_inventario() crea una Lista y pide los registros a persistencia.leer_productos().
3. Cada registro se valida con crear_producto() y se inserta mediante inserta1().
4. El servidor conserva esa misma lista para atender las solicitudes. No vuelve a leer toda la base en cada GET.

Si no hay registros, se crea un inventario vacío. Si un registro persistido no cumple las reglas, la carga propaga el error en lugar de omitirlo silenciosamente.

## 7. Consultar productos desde la pantalla

1. app.js ejecuta fetch("/api/productos").
2. do_GET() reconoce la ruta y llama a consultar_productos().
3. barrido() obtiene los elementos y la lógica devuelve copias.
4. responder_json() serializa los datos y devuelve HTTP 200.
5. JavaScript construye las filas y calcula los indicadores de stock.

La búsqueda por nombre o ID se realiza sobre la copia de productos recibida en JavaScript. No consulta SQLite por cada tecla ni modifica el orden del TDA. Los nombres se insertan con textContent para mostrarlos como texto.

## 8. Agregar un producto

1. El usuario completa el formulario; el navegador y JavaScript verifican los datos.
2. Se envía POST /api/productos con nombre, cantidad_disponible y precio.
3. El servidor comprueba el formato del pedido y llama a agregar_producto().
4. La lógica vuelve a validar los datos, independientemente del frontend.
5. insertar_producto() guarda el registro y devuelve el ID generado por SQLite.
6. El producto recibe ese ID y se incorpora a la lista con inserta1().
7. El servidor responde HTTP 201 con el diccionario creado.
8. La pantalla limpia el formulario, quita el filtro y vuelve a consultar la API.

## Ejemplo de alta

Si el último ID es 20, un alta podría recibir el ID 21. Un producto con nombre "Mouse USB", cantidad 5 y precio 12000 se guarda en SQLite y queda al final de la lista. El ID no se escribe en el formulario y los huecos de IDs no se rellenan necesariamente.

Si SQLite falla, la función no llega a insertar en memoria. Si la respuesta se pierde después de guardar, el usuario debe actualizar el catálogo antes de reenviar, porque el alta podría haberse realizado.

# Edición y eliminación

## 9. Editar desde el catálogo

Cada fila tiene un botón Editar. iniciarEdicion() copia sus valores al formulario, guarda el ID en idProductoEnEdicion, muestra Cancelar edición y cambia el botón principal a Guardar cambios.

Al enviar, guardarNuevoProducto() distingue el modo: si el ID es null usa POST; si hay un ID usa PUT /api/productos/ID. El cuerpo contiene los tres campos editables; el identificador viaja en la dirección.

En el backend, modificar_producto() busca el producto en el TDA, valida los datos y actualiza SQLite. Si se modificó el registro, elimina la versión anterior con eliminar1() e inserta la nueva con inserta1(). El ID se conserva y el producto mantiene su posición por ese criterio.

Ante un ID inexistente la ruta devuelve HTTP 404. Ante datos inválidos devuelve 400. Si el guardado se confirma, devuelve 200, la interfaz vuelve al modo alta y recarga el catálogo. Cancelar edición limpia el formulario sin enviar cambios al servidor.

### Ejemplo de modificación

El producto #3 tiene 10 unidades y precio 35000. Al cambiarlo a 15 unidades y precio 40000, el mismo ID #3 conserva su identidad. Los datos actualizados quedan en SQLite y en la lista; no se crea otro producto.

## 10. Eliminación implementada en la lógica

eliminar_producto() consulta primero el TDA. Si el ID existe, intenta borrar en SQLite y solo después retira el producto con eliminar1(). Devuelve una copia de lo eliminado o None cuando no se encuentra.

Si una clave foránea impide borrar un producto asociado a compras o esperas, SQLite lanza IntegrityError. Como el error ocurre antes de retirar el nodo, el inventario permanece sin cambios.

**Pendiente:** exponer esta operación mediante DELETE /api/productos/ID y agregar la acción correspondiente al frontend. La existencia de eliminar_producto() no significa que ya se pueda eliminar desde el navegador.

## 11. Operaciones disponibles por HTTP

| Método y ruta | Función | Éxito |
|---|---|---|
| GET /api/productos | Consultar el inventario completo. | 200 |
| POST /api/productos | Registrar un nuevo producto. | 201 |
| PUT /api/productos/ID | Actualizar los tres campos editables. | 200 |
| DELETE /api/productos/ID | Todavía no implementado en la API. | Pendiente |

# Persistencia, control y comprobación

## 12. Cómo se conservan los registros

persistencia.py resuelve BD/tablas.db desde la ubicación del propio archivo, abre una conexión para cada operación, activa foreign_keys y usa consultas parametrizadas. Las escrituras usan with conexion para confirmar o revertir la operación SQL; finally cierra la conexión.

Al detener el programa, la lista desaparece de memoria, pero SQLite conserva los registros. En el siguiente inicio cargar_inventario() reconstruye la lista a partir de la base. Una modificación externa de tablas.db mientras el servidor está abierto no actualiza automáticamente la lista: el botón Actualizar consulta la memoria del servidor.

## 13. Concurrencia y errores

ThreadingHTTPServer puede atender varios pedidos en hilos diferentes. Un Lock compartido protege las consultas y las escrituras del inventario desde GET, POST y PUT para que no se intercalen esas operaciones dentro del mismo servidor.

POST y PUT exigen application/json, un cuerpo declarado de hasta 65536 bytes y exactamente los campos nombre, cantidad_disponible y precio. Los errores de datos reciben 400, los tipos de contenido incorrectos 415, las rutas o productos ausentes 404 y los errores de SQLite 500.

**Límite de consistencia:** el bloqueo no convierte a SQLite y al TDA en una única transacción. Si el proceso falla después de confirmar SQL y antes de actualizar la lista, puede existir una diferencia temporal; reiniciar reconstruye el inventario desde SQLite. Actualmente no hay recuperación automática de ese caso ni sincronización entre varios procesos.

## 14. Qué se comprobó

El usuario confirmó las pruebas temporales de alta, modificación y eliminación: persistencia, recuperación al recargar, datos inválidos, IDs inexistentes y eliminación bloqueada por claves foráneas. También confirmó la consulta web y el alta desde la pantalla.

Para la interfaz de edición se verificaron sintaxis y lógica con DOM y API simulados: botón Editar, envío PUT, cancelación, conservación del modo ante error y alta POST. Esa verificación no sustituye una comprobación manual de edición contra SQLite desde el navegador.

## 15. Guía breve de uso y prueba manual

```
Desde la raíz TIENDA_TP: python -B main.py
Pantalla: http://127.0.0.1:8000/
Consulta JSON: http://127.0.0.1:8000/api/productos
```

Agregar: completar y guardar; comprobar tabla y resumen. Editar: seleccionar una fila, cambiar datos y guardar; comprobar que el ID y la cantidad de productos no cambien. Reiniciar el servidor y verificar que los valores persistan. Las operaciones desde la pantalla escriben en la base real.

**Alcance del informe:** describe el código actual del módulo de inventario. No implementa nuevas funciones ni cambia los registros de la tienda. La carpeta .git no se utiliza para generar este documento.